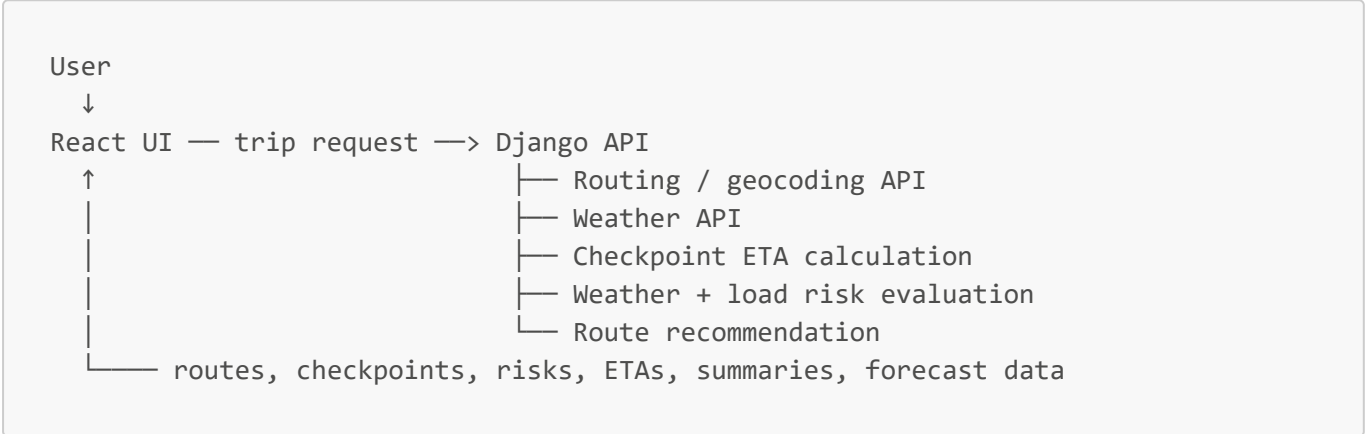
Weather-Aware Truck Routing — Architecture

Project: Spotter.ai Full Stack Developer Assessment
Stack required by brief: React + Django

1. Architecture Overview

Use a small client–server application. React handles trip input and visualization; Django coordinates routing and weather data, evaluates checkpoints, and ranks routes. Third-party services provide routing, geocoding, and weather data. A custom routing engine is not required.



2. Main Components

- **React UI:** collects origin, destination, departure date/time, and load weight; displays route options, summaries, ETAs, checkpoint risk levels, and the map.
- **Django API:** validates the trip request and coordinates the analysis. Keep provider integration, checkpoint generation, risk evaluation, and route ranking in separate service modules.
- **Routing/geocoding integration:** resolves locations and requests route alternatives and route geometry from a third-party provider.
- **Checkpoint and ETA service:** creates checkpoints at the selected 10-, 25-, or 50-mile interval and estimates arrival time at each checkpoint.
- **Weather integration:** obtains weather values for each checkpoint at its estimated arrival time and provides the data needed for the 0–48-hour forecast heatmap.
- **Risk engine:** applies the assessment's weather thresholds and load rules to checkpoint weather.
- **Recommendation service:** compares routes in the specified order: fewest Severe miles, fewest High miles, lowest average risk, then shortest travel time.
- **Map visualization:** renders route geometry, checkpoints, risk levels, route summaries, and the weather heatmap with a 0–48-hour slider.

3. Request-to-Result Flow

1. The user submits the trip details and checkpoint interval.
2. React sends the request to Django.
3. Django validates the inputs and requests route alternatives.
4. For each route, Django generates checkpoints and estimates checkpoint arrival times.

5. Django obtains weather data for each checkpoint and evaluates weather/load risk.
6. Django calculates route summaries and ranks eligible routes using the assessment's preference order.
7. React displays the route results and map layers. The forecast slider updates the heatmap's selected forecast time.

4. Key Data Passed Between Components

- **Trip request:** origin, destination, departure date/time, load weight, checkpoint interval.
- **Route:** route ID, geometry, distance, estimated duration, ETA, and summary metrics.
- **Checkpoint:** location/coordinates, distance along route, estimated arrival time, weather measurements, risk level, and explanatory warnings where applicable.
- **Analysis result:** route options, recommendation, checkpoint results, and warnings for unavailable or incomplete data.

5. Implementation Notes

- Keep the weather/load rules and route-ranking logic independent of React so they can be tested separately.
- Keep third-party API credentials on the backend; do not expose private keys in the React bundle.
- Handle provider errors and missing forecast data explicitly. Do not silently treat missing weather as Low risk or invent route options if fewer than three are returned.
- The brief does not define several behaviors, including threshold-boundary interpretation, combining weather risks, conversion of checkpoint risk to risk miles, No Travel route eligibility, and the slider's reference time. Document the chosen implementation behavior rather than presenting it as a stated requirement.
- A database is not specified by the brief. Do not add persistence unless implementation needs demonstrate a clear benefit.

6. Scope

Prioritize the route/weather/risk/recommendation workflow and smooth map/heatmap interaction, which together account for 75% of the assessment score. Keep the solution within the stated maximum of four days and 16 work hours. Deliver a GitHub repository, hosted application, and 5–10 minute Loom walkthrough.